Міністерство освіти і науки України

Національний університет «Львівська політехніка»

Кафедра СКС

Звіт

до лабораторної роботи №3

з дисципліни: “Дослідження і проєктування надвеликих інтегральних схем

на тему: “Розробка конвеєрної структури спеціалізованого перемножувача на константу з оптимізацією.”

Варіант №12

Виконав студент групи КІСК-13:

Романів В. А.

Перевірив: д.н.т., професор

Клим Г. І.

Львів 2024

Мета: згідно заданого варіанту розробити конвеєрну структуру перемножувача на константу з оптимізацією, та за допомогою мови VHDL описати розроблену структуру пристрою та провести функціональну симуляція проекту.

Послідовність виконання роботи

1. Ознайомитись з викладеним вище теоретичним матеріалом.
2. Нарисувати структуру конвеєризованого перемножувача на константу з оптимізацією згідно заданого варіанту.
3. За допомогою мови VHDL описати розроблену схему пристрою.
4. Провести моделювання на функціональному рівні розробленого VHDL - проекту.
5. Скласти звіт та захистити його.

Дані згідно варіанту

Таб. 1

Значення константи згідно варіанту

№	**2-ове представлення константи**	**16-ове представлення константи**
12	0,01011011001110110000011100001010	5B3B070A

Оптимізована константа

Користуючись правилом A*B=A*(B-1)+A, і позначенням 1 = -1, мінімізуємо кількість одиниць в константі.

1. 01011011001110110000011100001010
2. 01011011001110110000100100001010
3. 01011011001111010000100100001010
4. 01011011010001010000100100001010

5. 0101110<u>1</u>0100<u>0</u><u>1</u>0000100<u>1</u>00001010

6. 0110<u>0</u><u>1</u>0100<u>0</u><u>1</u>0000100<u>1</u>00001010

Таким чином, оптимізована константа рівна - 0, 0110 0<u>1</u>0<u>1</u> 0100 0<u>1</u>0<u>1</u> 0000 100<u>1</u> 0000 1010.

Y = x.L1 + x.L3 - x.L8 + x.L11 - x.L16 - x.L18 + x.L22 - x.L24 - x.L26 + x.L29 + x.L30 = (x.L1 + x.L3) + (x.L11 + x.L22) + (x.L29 + x.L30) - ((x.L8 + x.L16) + (x.L18 + x.L24) + x.L26)

Хід виконання

1. **Ознайомитись з викладеним вище теоретичним матеріалом.**

В матричних і деревовидних структурах паралельних перемножувачів закладений ще один потенціал підвищення продуктивності – можливість конвеєризації. При конвеєризації весь процес обчислення розбивається на послідовність закінченних кроків. Кожний з етапів процедури множення виконується на своїй сходинці конвеєра, при чому всі сходинки конвеєра працюють паралельно. Результати, отримані на - ій сходинці, передаються на подальшу обробку в - у сходинку конвеєра. Перенос інформації з однієї сходинки на іншу відбувається через буферну пам’ять, яка розміщена між сходинками конвеєра. На рис.1. зображено структуру конвеєрного перемножувача.

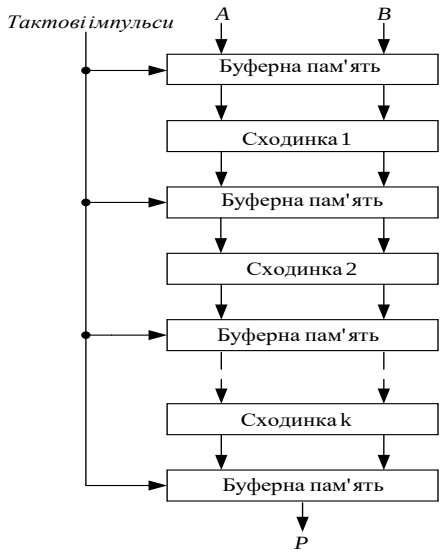

Рис. 1. Структура конвеєрного перемножувача.

Результат виконаної операції на сходинці заноситься в буферну пам’ять і дана сходинка може приступати до обробки наступної порції даних операцій, в той час, як чергова сходинка конвеєра в якості вхідних даних, використовує дані, які зберігаються в буферній пам’яті на її виході. Синхронність роботи конвеєра забеспечується тактовими імпульсами, період яких τ визначається самою найповільнішою сходинкою конвеєра τ_i і затримкою в елементі буферної пам’яті $\tau_l : \tau = \max(\tau_1, \tau_2, ..., \tau_k) + \tau_i$. В конвеєрному перемножувачі із k сходинок перемножувані дані можуть подаватися на вхід з інтервалом в k раз меньшим, ніж в випадку звичайного перемножувача. І в тому ж темпі появляються результати на виході.

Схема конвеєра легко може бути застосована до перемножувача на константу, який має деревовидну структуру. На рис.2. зображена оптимізована структура конвеєрного перемножувача для заданої константи, яка розглядалася вище.

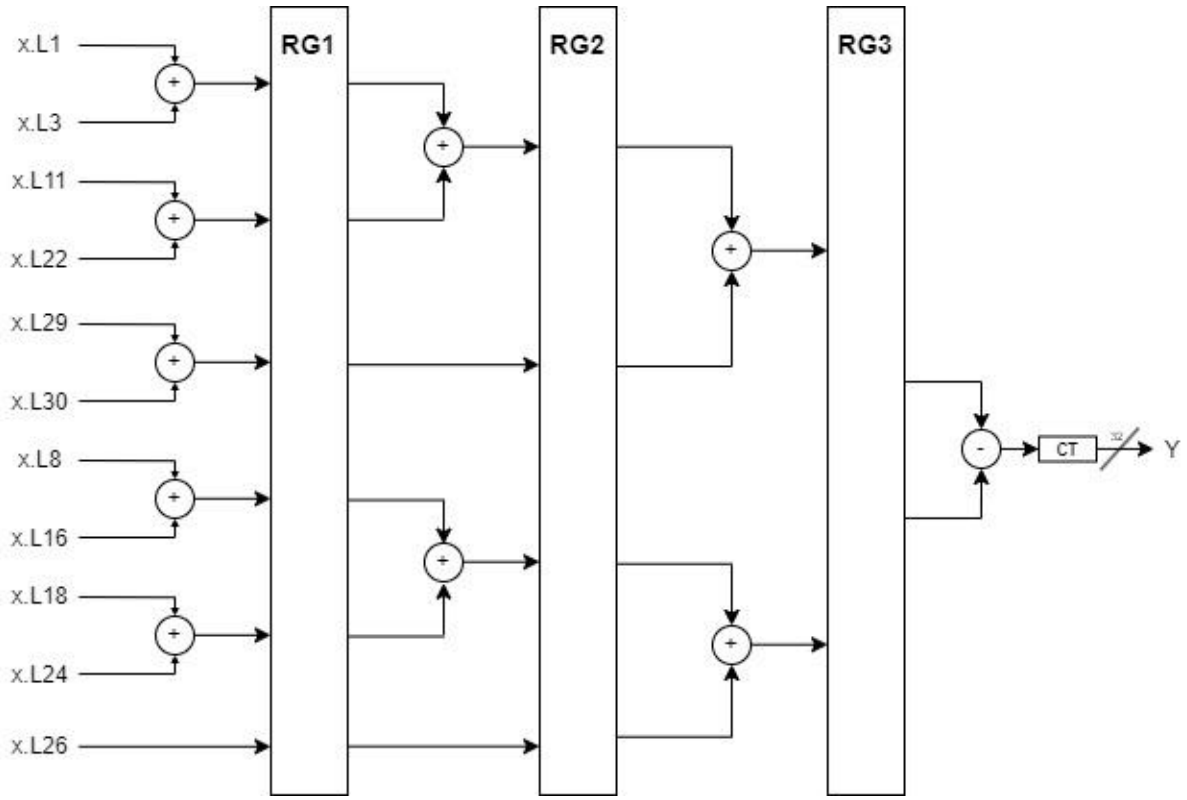

Рис. 2. Структурна схема конвеєризованого перемножувача на константу з оптимізацією.

В даному випадку в якості буферної памяті застосовуються регістри (RG1, RG2, RG3).

Нижче подано vhdl-код конвеєризованої структури перемножувача на константу з оптимізацією:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

entity multiple_conv is
    port(
        DIN : in STD_LOGIC_VECTOR(31 downto 0);
        CLK : in STD_LOGIC;
        RST : in STD_LOGIC;
        DOUT : out STD_LOGIC_VECTOR(31 downto 0)
    );
end multiple_conv;


architecture multiple_conv of multiple_conv is
```

```
        signal X_L1, X_L3, X_L8, X_L11, X_L16, X_L18, X_L22, X_L24, X_L26, X_L29,
X_L30 : STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
        signal   X1_0_in,   X1_1_in,   X1_2_in,   X1_3_in,   X1_4_in,   X1_5_in   :
STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
        signal       X1_0_out,X1_1_out,X1_2_out,X1_3_out,X1_4_out,X1_5_out       :
STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
        signal X2_0_in, X2_1_in, X2_2_in, X2_3_in : STD_LOGIC_VECTOR(63 downto 0)
:= (others => '0');
        signal X2_0_out, X2_1_out, X2_2_out, X2_3_out : STD_LOGIC_VECTOR(63 downto
0) := (others => '0');
        signal X3_0_in,X3_1_in : STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
        signal X3_0_out,X3_1_out : STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
        signal X4_0 : STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
    begin
    -- X with shift
        X_L1  <= "00000000000000000000000000000000" & DIN & "0";
        X_L3  <= "0000000000000000000000000000000" & DIN & "000";
        X_L8  <= "00000000000000000000000" & DIN & "00000000";
        X_L11 <= "000000000000000000000" & DIN & "00000000000";
        X_L16 <= "0000000000000000" & DIN & "0000000000000000";
        X_L18 <= "00000000000000" & DIN & "000000000000000000";
        X_L22 <= "0000000000" & DIN & "0000000000000000000000";
        X_L24 <= "00000000" & DIN & "00000000000000000000000";
        X_L26 <= "000000" & DIN & "0000000000000000000000000";
        X_L29 <= "000" & DIN & "00000000000000000000000000000";
        X_L30 <= "00" & DIN & "000000000000000000000000000000";

    -- stage 1 ------------------------------------------------------------
        X1_0_in <= X_L1 + X_L3;
        X1_1_in <= X_L11 + X_L22;
        X1_2_in <= X_L29 + X_L30;
        X1_3_in <= X_L8 + X_L16;
        X1_4_in <= X_L18 + X_L24;
        X1_5_in <= X_L26;
    -- stage 2 ------------------------------------------------------------
        X2_0_in <= X1_0_out + X1_1_out;
        X2_1_in <= X1_2_out;
        X2_2_in <= X1_3_out + X1_4_out;
        X2_3_in <= X1_5_out;
    -- stage 3 ------------------------------------------------------------
        X3_0_in <= X2_0_out + X2_1_out;
        X3_1_in <= X2_2_out + X2_3_out;
    -- stage 4 ------------------------------------------------------------
        X4_0 <= X3_0_out - X3_1_out;
    --------------------------------
        DOUT <= X4_0 (63 downto 32);

    --Reg file 1----------------------------------------------------------
        process (CLK, RST)
        begin
            if RST='1' then
                X1_0_out <= (others => '0');
                X1_1_out <= (others => '0');
                X1_2_out <= (others => '0');
                X1_3_out <= (others => '0');
```

```
            X1_4_out <= (others => '0');
            X1_5_out <= (others => '0');
        elsif (CLK'event and CLK='1') then
            X1_0_out <= X1_0_in;
            X1_1_out <= X1_1_in;
            X1_2_out <= X1_2_in;
            X1_3_out <= X1_3_in;
            X1_4_out <= X1_4_in;
            X1_5_out <= X1_5_in;
        end if;
    end process;
--Reg file 2-----------------------------------------------------------
    process (CLK, RST)
    begin
        if RST='1' then
            X2_0_out <= (others => '0');
            X2_1_out <= (others => '0');
            X2_2_out <= (others => '0');
            X2_3_out <= (others => '0');
        elsif (CLK'event and CLK='1') then
            X2_0_out <= X2_0_in;
            X2_1_out <= X2_1_in;
            X2_2_out <= X2_2_in;
            X2_3_out <= X2_3_in;
        end if;
    end process;
--Reg file 3-----------------------------------------------------------
    process (CLK, RST)
    begin
        if RST='1' then
            X3_0_out <= (others => '0');
            X3_1_out <= (others => '0');
        elsif (CLK'event and CLK='1') then
            X3_0_out <= X3_0_in;
            X3_1_out <= X3_1_in;
        end if;
    end process;

end multiple_conv;
```

На рис. 3 зображена функціональна діаграма роботи конвеєризованого перемножувача на константу з оптимізацією.

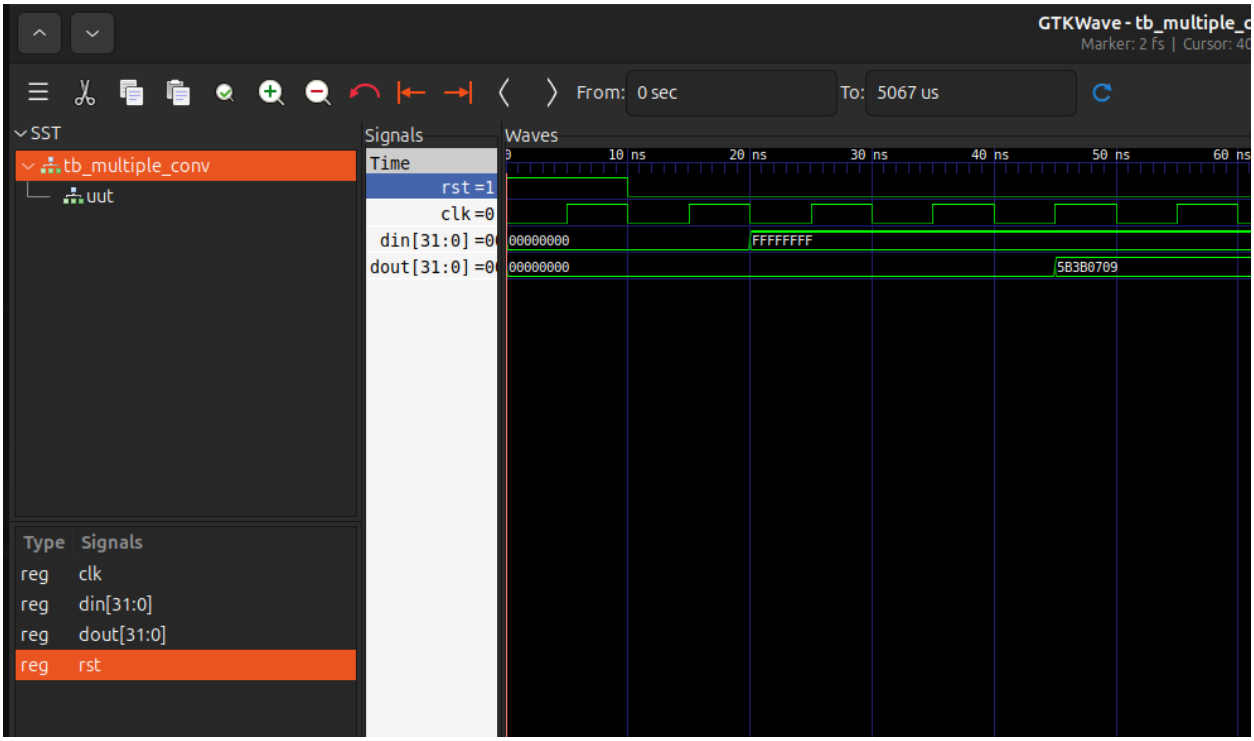

Рис. 3. Функціональна діаграма роботи конвеєризованого перемножувача на константу з оптимізацією.

На даній діаграмі можна бачити, що з моменту подачі на вхід шістнадцяткового числа FFFFFFFF, результат на виході DOUT появляється на виході третього такту роботи нашого пристрою по зростаючому фронту синхроімпульсу (тобто через три сходинки конвеєра).

Висновок: під час виконання даної лабораторної роботи мною було розроблено конвеєрну структуру перемножувача на константу з оптимізацією згідно заданого варіанту, та за допомогою мови VHDL описано розроблену структуру пристрою та виконано функціональну симуляція проекту.